

RP2350 SDR CW Interface

Author: J. Maas, PA3GSB

Document version: 1.1

Firmware program version: 0.1

Revision date: 9 August 2026

Design origin and goals

A CW-related question on the RadioBerry forum led to the development of this project.

Fast CW operators who rely on sidetone can experience latency between pressing the key or paddle and actually hearing the sidetone. Even a relatively small delay can become noticeable and distracting at higher CW speeds.

While looking for a way to solve this problem, I came across an excellent proposal by Steve Haynal, KF7O. This project therefore takes its architectural starting point from his January 2021 proposal, [Low Latency CW Sidetone](#), published on the Hermes-Lite mailing list.

Steve proposed combining a CW keyer and a class-compliant USB sound card in a single inexpensive external device. The important operator-feedback path can then remain entirely local: a paddle event generates the sidetone directly inside the adapter, where it can be mixed with, or replace, the receive audio coming from the computer.

This avoids routing the sidetone through the PC and its USB and audio buffering, thereby eliminating an important source of latency.

Keeping the keyer logic, low-speed audio processing, buffering, and control functions inside a small microcontroller also leaves the Hermes-Lite FPGA resources available for what they are best suited for: RF and DSP processing.

Steve explicitly mentioned the Raspberry Pi Pico as a possible low-cost platform for such an implementation. This project follows that direction using its more powerful successor, the **RP2350**, together with a **WM8960 audio codec**.

An AI-Driven Development Experiment

There was another goal behind this project as well.

I wanted to find out how far I could take the development using modern AI coding tools.

My intention was deliberately **not to write the implementation myself**. Instead, I wanted to describe what I wanted, discuss problems with the AI, provide measurements and debug output, and let the AI generate and modify the code based on those prompts.

Development was done in Visual Studio Code. I initially started using GitHub Copilot, but later moved to OpenAI Codex for most of the development work.

The remarkable result is that the functionality in this project was created through interaction with AI. I did not manually program the implementation line by line. My role was primarily to define the requirements, connect and test the hardware, observe the results, provide logs and measurements, and tell the AI what should be changed next.

In other words, the project became an experiment in both **low-latency CW hardware** and **AI-assisted engineering**.

For me, that has been one of the most impressive parts of the project: seeing an idea progress from a discussion and an architectural concept to working RP2350 firmware, USB audio, I2S audio, WM8960 control, and CW functionality largely through a conversation with an AI coding system.

A Note About Support and Using AI

This project is published as an experiment and as a reference for others who may want to build or extend something similar. I do not intend to provide individual support for the code or for adapting it to other hardware or configurations.

Fortunately, there is now another option:

Ask an AI.

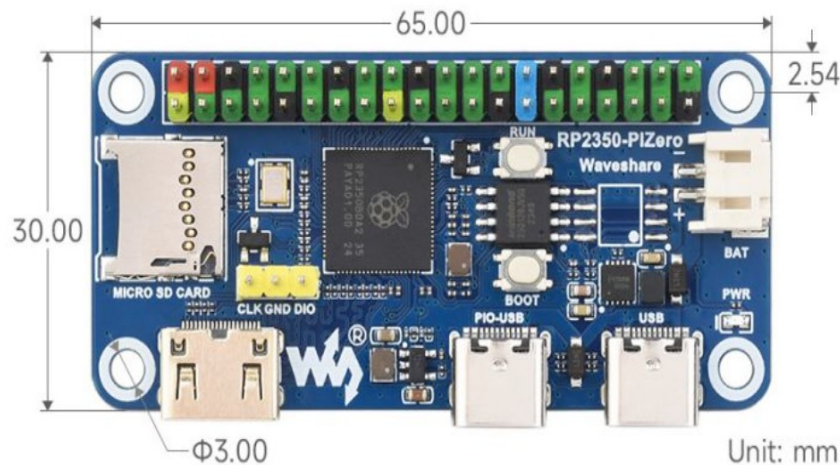
If something does not compile, if you want to use different GPIO pins, another codec, change the USB implementation, add another feature, or simply want to understand how part of the code works, give the source code, the error messages, the hardware documentation, and your requirements to an AI coding assistant.

That is, after all, essentially how this project was built.

In that sense, this manual may sometimes feel almost unnecessary: instead of searching through every page, a user can give this document, README.md, the source code, and any error messages to an AI assistant and ask a question in ordinary language.

The manual is still important. It provides the verified project-specific facts that an AI needs as context. AI answers should be checked against the current README, source code, wiring table, and hardware documentation, especially before changing connections or applying power.

RP2350-PiZero



The **Waveshare RP2350-PiZero** is a compact microcontroller board that has a similar shape and 40-pin connector to a Raspberry Pi Zero, but technically it is much closer to a **Raspberry Pi Pico 2**.

It does **not run Linux**. Instead, you program it directly using for example C/C++ with the Pico SDK, MicroPython, or Arduino.

The board is based on the **Raspberry Pi RP2350B microcontroller**. It can use either two ARM Cortex-M33 cores or two Hazard3 RISC-V cores, running at up to around **150 MHz**. It also has **520 KB of SRAM** and Waveshare includes **16 MB of flash memory** on the board.

The RP2350 itself also provides interfaces such as **SPI, I²C, UART, PWM and 12 PIO state machines**.

The PIO blocks are especially useful because they allow you to implement very precise hardware protocols yourself. For example, they can be used for **I²S audio, additional USB interfaces, custom serial protocols, or very accurate timing**.

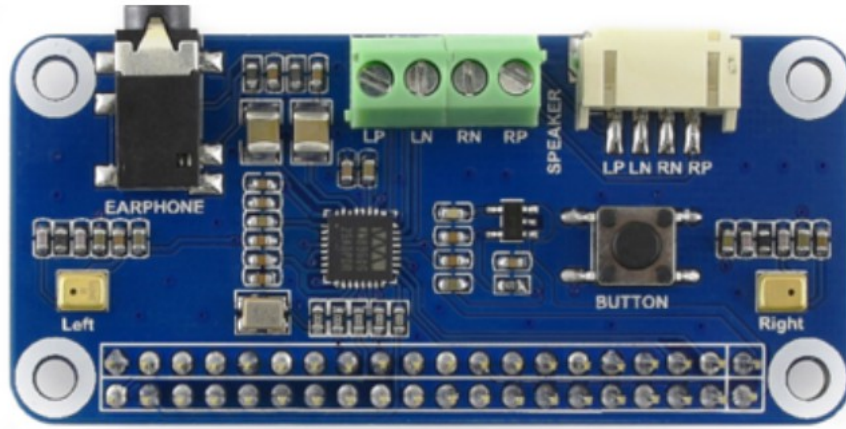
A simple way to describe the board is:

RP2350-PiZero = Pico 2-like microcontroller + Raspberry Pi Zero form factor + many extra interfaces.

The main difference compared with a normal Raspberry Pi Zero is that the RP2350-PiZero is a **microcontroller**, not a small Linux computer.

Raspberry Pi Zero	RP2350-PiZero
Linux computer	Microcontroller
Boots from SD card	Runs firmware from flash
Much more RAM	520 KB SRAM
GHz-class CPU	Around 150 MHz
Linux-based peripherals	Direct hardware control
Less deterministic timing	Very good real-time timing
This makes the RP2350-PiZero especially suitable for embedded and real-time projects where precise control of hardware is important.	

WM8960 Audio HAT



The **Waveshare WM8960 Audio HAT** is an audio expansion board based on the **WM8960 stereo audio codec**. It can be connected to the **Waveshare RP2350-PiZero** to add audio input and output capabilities to the board.

The WM8960 converts between **digital and analog audio**. It contains both a **DAC (Digital-to-Analog Converter)** for audio playback and an **ADC (Analog-to-Digital Converter)** for recording audio.

Communication between the **Waveshare RP2350-PiZero** and the WM8960 Audio HAT is mainly handled through two interfaces:

- **I²C** is used to configure the WM8960, such as volume levels, audio routing, input/output selection and audio format.
- **I²S** is used to transfer the actual digital audio samples between the RP2350 and the WM8960.

The WM8960 Audio HAT provides several useful audio functions, including:

- Stereo headphone output
- Stereo speaker output
- Microphone input
- Digital I²S audio interface
- I²C control interface
- Adjustable headphone and speaker volume
- Integrated audio amplification

A simple way to describe the combination is:

Waveshare RP2350-PiZero + WM8960 Audio HAT = microcontroller + digital audio interface + audio converter + amplifier.

For audio playback, the **Waveshare RP2350-PiZero** generates or processes digital audio samples and sends them over I²S to the WM8960. The WM8960 converts these digital samples into an analog signal that can be played through headphones or speakers.

For audio recording, the WM8960 can take an analog microphone signal, convert it into digital audio samples and send those samples back to the **Waveshare RP2350-PiZero**.

Because the **Waveshare RP2350-PiZero** can generate and receive I²S signals using the RP2350's **PIO hardware**, it provides accurate and predictable audio timing without requiring an operating system such as Linux.

Required hardware

The basic interface consists of a Waveshare RP2350-PiZero, a Waveshare WM8960 Audio HAT, and a USB cable that supports both power and data.

- Optional controls include a CW paddle, a separate straight key, a KY-040 rotary encoder module, five function LEDs, and two output-status LEDs.
- Fit one 220–470 ohm current-limiting resistor for every LED.
 - Power an encoder module from 3.3 V only. Never apply 5 V signals to an RP2350 GPIO pin.

Hardware connections

All key and encoder inputs are active low and use internal pull-up resistors. Connect their common contacts to GND.

- Paddle DIT: GPIO23, physical pin 16; contact to GND.
 - Paddle DAH: GPIO27, physical pin 13; contact to GND.
 - Straight key: GPIO22, physical pin 15; contact to GND.
 - Encoder A/CLK: GPIO15, physical pin 29.
 - Encoder B/DT: GPIO6, physical pin 31.
 - Encoder button: GPIO13, physical pin 33; contact to GND.
- The WM8960 Audio HAT reserves GPIO2, GPIO3, GPIO17, GPIO18, GPIO19, GPIO20, and GPIO21. Consult README.md for the complete GPIO and LED connection table.

Installing the firmware

1. Disconnect USB.
2. Hold BOOTSEL while reconnecting USB, then release it.
3. Open the removable drive named RPI-RP2 or similar.
4. Copy rp2350-sdr-cw-interface.uf2 to the drive.
5. The board restarts automatically as the RP2350 SDR Interface.

Windows may assign different COM-port numbers after an update. Check Device Manager again.

First-use checklist

- Check that both RP2350 SDR audio devices are visible.
- Check that two serial ports are visible.
- Open RP2350 SDR Console and enter help or show.
- Connect WKdemo to RP2350 SDR WinKey and confirm WinKey2 V23 OnLine.
- Play test audio and test DIT, DAH, straight key, and sidetone.

Operation on Windows

Connect the RP2350 SDR Interface to the Windows PC using a suitable USB data cable. Windows should automatically detect the interface and install the standard USB drivers.

After USB enumeration, the interface provides the following USB functions:

- **RP2350 SDR RX Audio** — a stereo, 48 kHz, 16-bit playback device. Audio sent by the PC to this device is reproduced through the WM8960 audio outputs.
- **RP2350 SDR TX Audio** — a mono, 48 kHz, 16-bit recording device. This transfers audio from the WM8960 input to the PC.
- **RP2350 SDR WinKey** — a serial COM port implementing the binary WinKey 2.3 protocol. This port is used by K1EL WKdemo and other WinKey-compatible applications.
- **RP2350 SDR Console** — a second serial COM port providing text-based commands, settings, status information, and decoded CW.
- **RP2350 SDR MIDI** — a separate USB MIDI device used for CW, PTT, and keyer-speed information. MIDI does not use a COM port.

No special Windows drivers should normally be required because the interface uses the standard USB Audio, CDC serial, and MIDI device classes supported by Windows.

The two serial ports can be found in:

Device Manager → Ports (COM & LPT)

Two COM ports belonging to the RP2350 SDR Interface should be visible. Note the COM-port number assigned to **RP2350 SDR WinKey**, as this is the port that must be selected in WKdemo.

Do not open the WinKey COM port with a regular serial terminal because it carries binary protocol data. For readable commands and status information, open **RP2350 SDR Console** instead. A baud rate such as 115200 may be selected, although the selected baud rate does not determine the actual USB transfer speed.

The audio interfaces can be checked under:

Settings → System → Sound

The following devices should be present:

- **RP2350 SDR RX Audio** under output devices;
- **RP2350 SDR TX Audio** under input devices.

Both audio devices use a sample rate of 48 kHz.

The MIDI interface appears as **RP2350 SDR MIDI** in compatible MIDI or amateur-radio software. It does not appear as a COM port.

Once both audio devices, both COM ports, and the MIDI device are visible, the RP2350 SDR Interface has been successfully detected by Windows and is ready for configuration and use.

Checking the COM ports

Open Windows **Device Manager** and expand **Ports (COM & LPT)**. When the RP2350 SDR Interface is detected correctly, two serial USB devices will be listed. In this example, Windows assigned **COM3** and **COM4**.

The two ports have different functions:

- one COM port provides the **RP2350 SDR WinKey** interface;
- the other provides the **RP2350 SDR Console** for settings, status information, and decoded CW.

The assigned COM-port numbers may differ between computers. Identify and note the WinKey COM-port number, as this is the port that must be selected in WKdemo or other WinKey-compatible software.

Windows may display both ports using the generic name **USB Serial Device**. If it is unclear which port is the WinKey interface, check the device properties or test the Console port with a serial terminal. Do not use the WinKey port as a regular text terminal because it carries binary protocol data.



Figure caption: The two COM ports of the RP2350 SDR Interface shown in Windows Device Manager.

CW Keyer Configuration with WKdemo

K1EL provides WKdemo for testing and configuring WinKey-compatible keyers. Use WKdemo for WinKeyer 1 or 2; WK3demo is not suitable for this WinKey 2.3-compatible interface.

Start WKdemo, select Setup, and choose the COM port assigned to RP2350 SDR WinKey. Do not select the RP2350 SDR Console port.

After the connection is opened, the status line should show WinKey2 V23 OnLine. This confirms that Windows, the USB serial connection, and the WinKey emulation are communicating correctly.

The RP2350 interface can therefore be operated much like a hardware K1EL WinKey device.

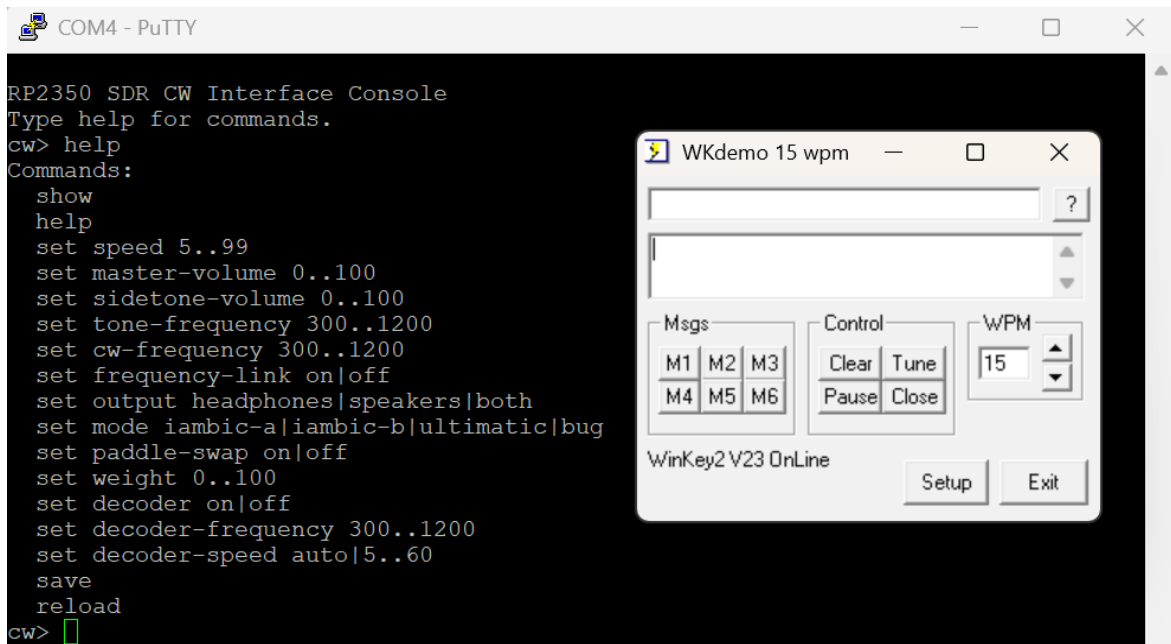
Using WKdemo, you can configure and test the CW keyer settings, including options such as:

- CW speed;
- paddle configuration;
- keying mode;
- weighting and timing;
- sidetone settings;
- PTT/keying behaviour;
- other WinKey-compatible configuration options supported by the firmware.

Changes made through WKdemo are sent directly to the keyer firmware running on the RP2350.

This also provides a convenient way to verify that the **USB serial interface and WinKey emulation are working correctly** before using the interface with other logging, contesting, or amateur-radio software.

The Console and WinKey interfaces can be used at the same time because they are separate COM ports. In the example below, PuTTY has opened RP2350 SDR Console on COM4, while WKdemo is connected to RP2350 SDR WinKey on COM3. Entering help in the Console displays the available text commands; WKdemo simultaneously reports WinKey2 V23 OnLine.



Select Setup in WKdemo to open the WinKeyer Setup window. Confirm that Comm Port is set to the RP2350 SDR WinKey port. Keyer mode, paddle swap, weighting, speed range, sidetone frequency, PTT behaviour, and other compatible parameters can then be adjusted. Select Done to apply the settings.

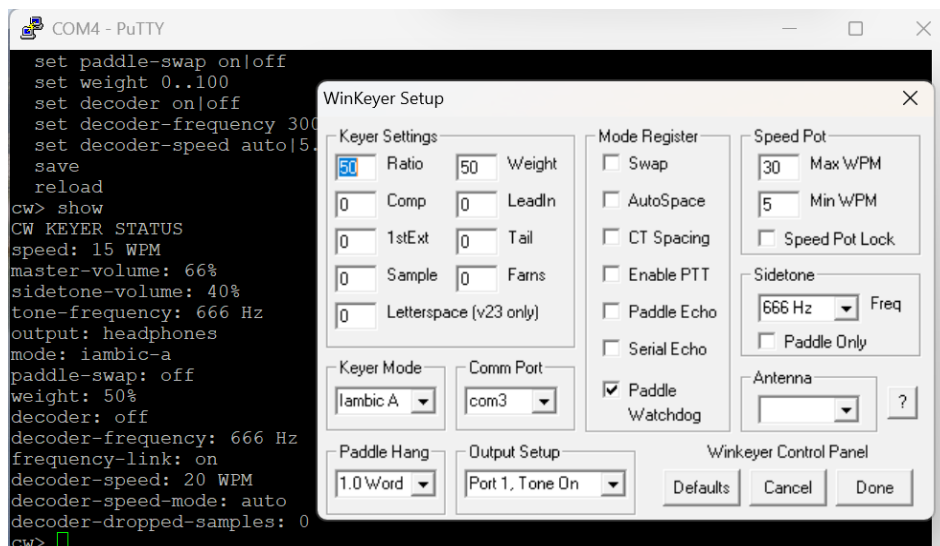
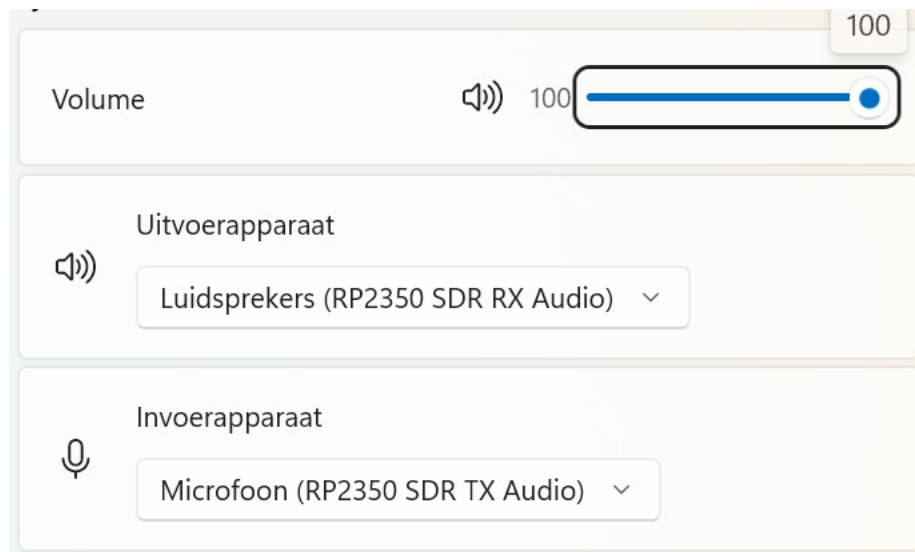


Figure: WKdemo WinKeyer Setup with COM3 selected for RP2350 SDR WinKey. The Console remains independently available on COM4.

Checking the audio devices

In Windows Settings, verify that Speakers (RP2350 SDR RX Audio) is selected as the output device and Microphone (RP2350 SDR TX Audio) as the input device. The names may be preceded by the localized Windows terms for speakers and microphone.



Operating the physical controls

At startup, rotating the encoder has no effect until a function is selected. Each short press selects the next function; rotate the encoder to change its value.

1. Master volume: 0–100%, steps of 5%.
2. Sidetone volume: 0–100%, steps of 5%.
3. Sidetone frequency: 300–1200 Hz, steps of 10 Hz.
4. Keyer speed: 5–99 WPM, steps of 1 WPM.
5. Output routing: headphones, speakers, or both.

Master volume affects received audio and sidetone. Sidetone volume affects only the locally generated CW tone. If the direction is reversed, swap GPIO15 and GPIO6.

Saving settings

Double-click the encoder button within 400 ms to save. A single click only selects the next function; disconnecting USB does not save automatically.

All seven LEDs illuminate during a write. One simultaneous pulse confirms success; three rapid pulses indicate an error. Save WKdemo changes with a double-click if they must survive a power cycle.

Paddle, straight key, and MIDI

The default mode is Iambic A. GPIO23 is DIT, GPIO27 is DAH, and GPIO22 is the separate straight-key input. The straight key has priority. WKdemo can select other supported modes and swap DIT and DAH.

RP2350 SDR MIDI uses channel 10. CW uses note 17, PTT uses note 18, and speed uses Control Change 3. MIDI is not a COM port.

Optional CW decoder

The firmware includes an optional real-time CW decoder. It listens to the RX or radio audio received from the computer before output muting, volume processing, and sidetone mixing. It does not decode audio from the WM8960 microphone input.

Decoded characters are printed on RP2350 SDR Console on a line beginning with CW:. After approximately two seconds without a detected tone, the line is closed and the `cw>` prompt returns.

The decoder is enabled by default and uses a single detection channel centred on 750 Hz. Match the radio or training-program pitch to this frequency, or select another centre frequency with `set decoder-frequency 300..1200`.

- `set decoder on` — enable decoding.
- `set decoder off` — disable decoding.
- `set decoder-frequency 300..1200` — select the expected CW tone in hertz.
- `set decoder-speed auto` — learn the CW speed automatically.
- `set decoder-speed 5..60` — use a fixed known character speed in WPM.

For Farnsworth material, enter the character speed rather than the lower effective speed. The `show` command reports decoder status, centre frequency, learned or fixed speed, and dropped samples.

CW decoding is an additional convenience rather than a replacement for operator copy or specialist decoding software. Results depend on correct pitch, signal-to-noise ratio, fading, interference, keying quality, and timing.

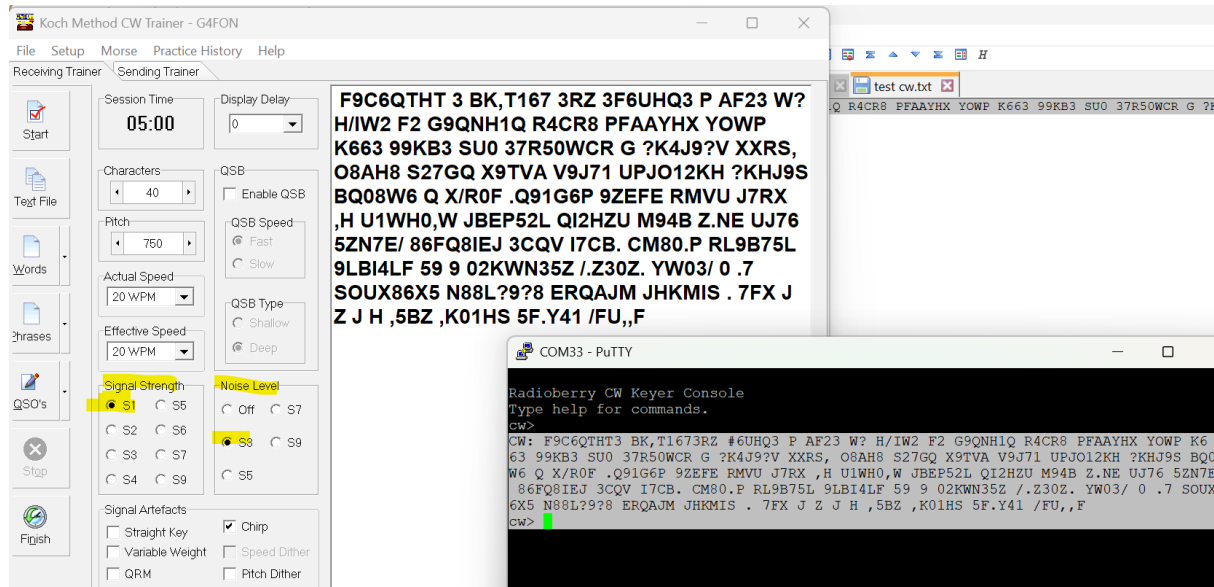


Figure: CW training audio decoded in real time and printed on the Console. This screenshot was made with an earlier firmware build that displayed the former Console name; current firmware displays RP2350 SDR CW Interface Console.

Troubleshooting

- USB not recognized: try another data cable, avoid an unpowered hub, fully disconnect the board, or reinstall the UF2 with BOOTSEL.
- Serial port busy: close WKdemo, terminal programs, logging software, and automatic serial monitors using that port.
- WKdemo does not connect: select RP2350 SDR WinKey, not RP2350 SDR Console. If necessary, use a COM number no higher than COM24.
- No audio: select RP2350 SDR RX Audio at 48 kHz, check the WM8960, increase master volume, and make sure the keyer is not permanently active.
- No sidetone: increase master and sidetone volume and test GPIO22, GPIO23, and GPIO27 by briefly connecting them to GND.
- Settings are lost: double-click the encoder and wait for the LED confirmation before removing power.

Further information

This document provides general installation and operating instructions. For the complete GPIO table, Console commands, current build instructions, technical details, and expanded troubleshooting information, consult README.md in the source-code repository.

Document version: 1.1. Author: J. Maas, PA3GSB. Revision date: 9 August 2026. Firmware program version: 0.1.